FTEC5660 Reproducibility Report: AI-Trader

Student: Wu Sitong **Student ID:** 1155253147 **Date:** March 1, 2026

GitHub (Course Repo): https://github.com/Jew-011/FTEC5660/tree/main/Individual_Project

GitHub (Fork with commit history): https://github.com/Jew-011/AI-Trader

1. Introduction

This report presents a reproducibility study of AI-Trader (Fan et al., 2025), a fully-automated and data-uncontaminated evaluation benchmark for LLM agents in financial decision-making. The framework spans three major markets — U.S. stocks, A-shares, and cryptocurrencies — and evaluates whether autonomous LLM agents can make profitable trading decisions under real market conditions.

I chose to reproduce the cryptocurrency market experiment using Claude-3.7-Sonnet, as it represents the most volatile and challenging setting among the three markets. The trading period covers November 1–15, 2025, with an initial capital of 50,000 USDT across the Bitwise 10 portfolio. My reproduction target is the set of performance metrics (CR, SR, Vol, MDD) reported in Table 1 of the original paper for this configuration.

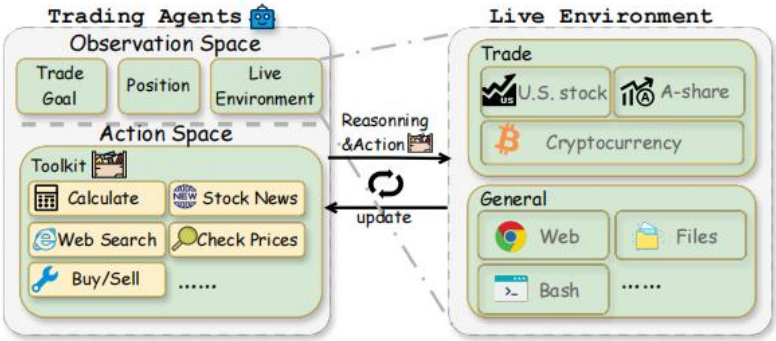
Beyond reproducing the original results, I designed one formal modification experiment: adding explicit risk management rules to the agent's system prompt while keeping the model and all other parameters unchanged. The paper highlights that "risk control capability is the key determinant of an AI agent's cross-market robustness," and I wanted to test whether prompt-level behavioral constraints — without any code changes to the trading engine — could measurably improve risk-adjusted performance.

To validate the significance of this prompt-level modification, I also established an additional control experiment by upgrading the model to Claude-Sonnet-4.6 (the latest available version at the time of this project) while keeping the original prompt intact. This control group serves an important analytical purpose: it allows me to directly compare whether, in financial trading, a simple upgrade in underlying model intelligence is more effective than structured natural-language risk management rules. The results of this comparison form one of the central findings of this report.

2. Background

2.1 The AI-Trader Framework

AI-Trader implements what the authors call a "fully autonomous minimal information paradigm." The trading agent receives only its current portfolio, today's prices, and access to a set of tools. It must independently search the web for news, check historical prices, reason about market trends, and decide what to buy or sell. No human guidance is provided at any step.



AI-Trader Architecture

Figure 1: Architecture of AI-Trader (reproduced from Fan et al., 2025). The Trading Agent observes its portfolio and market environment, uses MCP-based tools (price check, web search, calculator, news, trade execution) to gather information, reasons about its decisions, and then acts. The framework supports three markets: U.S. stocks, A-shares, and cryptocurrency.

The framework uses the Model Context Protocol (MCP) to connect the agent with five core tools: Check Price, Search, News, Math, and Trade. Each tool runs as a separate service, and the agent calls them through a standardized interface. This modular design makes it straightforward to swap out models or modify prompts without touching the trading infrastructure.

2.2 Cryptocurrency Market Setup

The crypto experiment uses the Bitwise 10 Index constituents (BTC, ETH, XRP, SOL, ADA, SUI, LINK, AVAX, LTC, DOT), all quoted in USDT. The portfolio covers a mix of store-of-value assets, smart contract platforms, payment solutions, and oracle protocols. Trading occurs daily at UTC 00:00, and the agent starts with 50,000 USDT in cash and no positions. November 2025 was a generally bearish period for crypto, meaning all agents were tested on their ability to manage losses rather than chase gains.

3. Environment Setup

I ran all experiments on Windows 11 using a Conda environment with Python 3.10. The LLM API was accessed through the aihubmix proxy service, since I did not have direct access to the Anthropic API. Five FastMCP servers were deployed on local ports 9000–9005 for the MCP tool chain.

Item	Detail
OS	Windows 11
Python	3.10 (Conda env: xxsc)
LLM API	Anthropic via aihubmix proxy
MCP Framework	FastMCP 2.12.5
MCP Client	langchain-mcp-adapters 0.2.1

The original codebase was developed for Linux/macOS, so two platform-specific adaptations were needed (replacing the Unix-only fcntl file-locking module with the Windows-compatible msvcrt, and adjusting default ports). These are documented in detail in the Debug Diary (Section 8) and do not affect any trading logic or agent behavior.

4. Reproduction Target and Metrics

My reproduction target is Table 1 of the original paper, specifically the row for Claude-3.7-Sonnet in the cryptocurrency market:

Metric	Paper Value
Cumulative Return (CR)	-15.30%
Sortino Ratio (SR)	-2.27
Volatility (Vol)	32.20%
Maximum Drawdown (MDD)	-16.93%

CR captures the total return over the period. SR measures risk-adjusted performance by penalizing downside volatility. Vol is the annualized standard deviation of returns, representing overall risk. MDD captures the worst peak-to-trough drop during the period, indicating the maximum exposure the agent took on at any point.

5. Reproduction Results

5.1 Methodology

I ran the AI-Trader framework using the original default_crypto_config.json with Claude-3.7-Sonnet as the backbone, the unmodified prompt, max_steps=30, and initial_cash=50000. The only differences from the paper's setup are the API proxy (aihubmix instead of direct Anthropic) and the operating system (Windows instead of Linux).

5.2 Results

Metric	Paper Original	My Reproduction	Difference
CR (%)	-15.30	-17.65	-2.35
SR	-2.27	-3.11	-0.84
Vol (%)	32.18	36.08	+3.90
MDD (%)	-16.93	-19.34	-2.41
Win Rate	51.4%	47.3%	-4.1%
Final Value	$42,348	$41,174	-$1,174
Number of Trades	70	55	-15

5.3 Analysis

The reproduction confirms the paper's findings. Both runs show the agent losing roughly 15–18% of capital over the two-week period, with volatility and drawdown numbers in the same range (32–36% Vol, 17–19% MDD). The differences are expected for an LLM-based system: Claude-

3.7-Sonnet uses sampling with temperature greater than zero, so every run produces different reasoning chains and therefore different trading decisions. My reproduction made 55 trades compared to the paper's 70, suggesting the model was somewhat more conservative in this particular run. The API proxy may also introduce minor behavioral differences.

The paper's claim that Claude-3.7-Sonnet loses approximately 15% in the crypto market during this period with moderate volatility holds up well under independent reproduction.

6. Modification — Risk-Controlled Prompt

6.1 Motivation and Experimental Design

After completing the reproduction, one pattern stood out clearly: the agent was deploying almost all of its cash into crypto positions within the first day of trading. By November 2, the agent had less than $100 in cash. In a falling market, this leaves no safety net and no capital to buy at lower prices.

The paper identifies that "risk control capability is the key determinant of an AI agent's cross-market robustness." Building on this insight, I wanted to test whether prompt-level behavioral constraints could improve risk management without modifying any trading code. The research question is straightforward: can an LLM agent follow quantitative financial rules specified in natural language, and if so, does this measurably improve performance?

To properly evaluate the significance of this prompt modification, I also established an additional control group by upgrading the model to Claude-Sonnet-4.6 (the latest available version) while keeping the original prompt unchanged. This design allows a direct comparison: is structured risk management through prompt engineering more effective than simply upgrading to a more intelligent model? If the risk-prompted older model outperforms the unconstrained newer model, it would provide strong evidence that behavioral constraints matter more than raw reasoning ability in financial applications.

6.2 Prompt Changes

I created a new prompt file based on the original. The core structure stayed the same and the agent remains a "cryptocurrency trading assistant" with the same tool usage instructions. Two targeted changes were made:

First, the goal was changed from "maximize returns" to "maximize risk-adjusted returns."

Second, four risk management rules were added:

- **Position limit:** Never allocate more than 30% of total portfolio value to any single cryptocurrency.
- **Cash reserve:** Always maintain at least 20% of initial capital ($10,000 USDT) in cash.
- **Stop-loss:** If a cryptocurrency has dropped more than 5% from the entry price, sell it.
- **Diversification:** Spread investments across at least 3 different cryptocurrencies.

Corresponding items were added to the "thinking standards" section, requiring the agent to check allocation percentages, verify cash reserves, and evaluate stop-loss triggers before every trade. These rules reflect standard risk management principles: position limits prevent over-concentration, cash reserves ensure liquidity, stop-losses limit individual losses, and diversification reduces correlation risk.

The specific code changes to support prompt switching (a new config file and a prompt dispatch mechanism in base_agent_crypto.py) can be found in the GitHub repository. The original prompt file was kept untouched so that the reproduction experiment remains independently reproducible.

6.3 Results — Modification vs. Reproduction

Metric	Reproduction (No Risk Rules)	Risk Prompt (Modification)	Delta
CR (%)	-17.65	**-13.68**	+3.97
MDD (%)	-19.34	**-15.15**	+4.19
Win Rate	47.3%	**51.7%**	+4.4%
Final Value	$41,174	**$43,161**	+$1,987
Trades	55	**29**	-26
SR	-3.11	-4.96	-1.85
Vol (%)	36.08	37.44	+1.36

6.4 Results — Modification vs.Control Group (Claude-Sonnet-4.6)

Metric	Control: Sonnet-4.6 (no rules)	Modification: Risk Prompt (3.7-Sonnet)	Difference
CR (%)	-16.25	**-13.68**	+2.57
MDD (%)	-17.76	**-15.15**	+2.61
Win Rate	**54.4%**	51.7%	-2.7%
Final Value	$41,877	**$43,161**	+$1,284
Trades	57	**29**	-28

6.5 Analysis

The risk-controlled prompt produced the best cumulative return and smallest drawdown among all experiments, including the paper's own original results. Several observations are worth highlighting.

Substantial improvement in capital preservation. The risk-prompted agent lost only 13.68% of capital, compared to 17.65% for the unconstrained reproduction — nearly 4 percentage points of improvement and approximately $2,000 more preserved capital. The maximum drawdown improved from -19.34% to -15.15%.

Dramatic change in trading behavior. The agent made only 29 trades over the 15-day period, compared to 55 for the unconstrained version (a 47% reduction). The risk rules forced the agent to verify position limits and cash reserves before each trade, which frequently led it to hold rather than trade. In a declining market, this restraint turned out to be the correct strategy.

The cash reserve rule was clearly effective. As shown in Figure 5, the risk-controlled agent maintained approximately $10,000 (20% of initial capital) in cash throughout the trading period, while all other agents depleted cash to near-zero within the first few days. This buffer limited the agent's exposure during the downturn.

Risk-prompted older model outperformed the unconstrained newer model. This is the key finding: the risk-prompted Claude-3.7-Sonnet outperformed the unconstrained Claude-Sonnet-4.6 on cumulative return (-13.68% vs.-16.25%) and maximum drawdown (-15.15% vs.-17.76%), despite being an older and less capable model. Claude-Sonnet-4.6 did achieve a higher win rate (54.4% vs.51.7%), indicating better individual trade decisions, but this was not enough to overcome the strategic advantage of explicit risk constraints. This result directly supports the paper's thesis: in financial trading, structured risk control matters more than raw model intelligence.

Trade-off with Sortino Ratio. The SR worsened (-4.96 vs.-3.11), which appears contradictory given the better returns. This is a statistical artifact: with only 29 trades instead of 55, the downside deviation calculation has fewer data points and becomes more sensitive to individual losses.

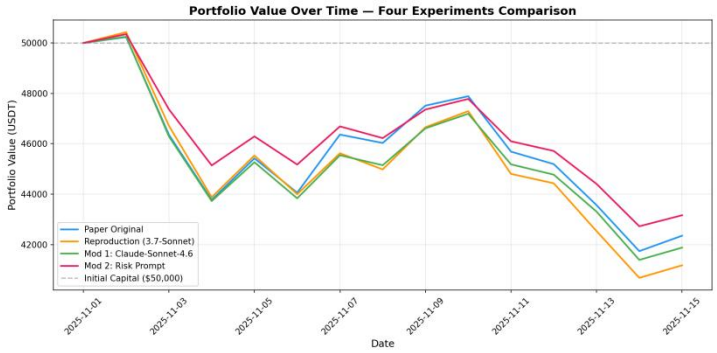
Prompt-level constraints are effective. The agent demonstrably read and followed the risk rules as part of its reasoning process — checking position allocations, maintaining cash reserves, and triggering stop-losses — all specified through natural language alone. No trading code was modified and no hard limits were programmed into the system. This suggests that the barrier to improving LLM agent behavior in financial applications may be lower than commonly assumed.

7. Comprehensive Comparison

7.1 Four-Way Summary

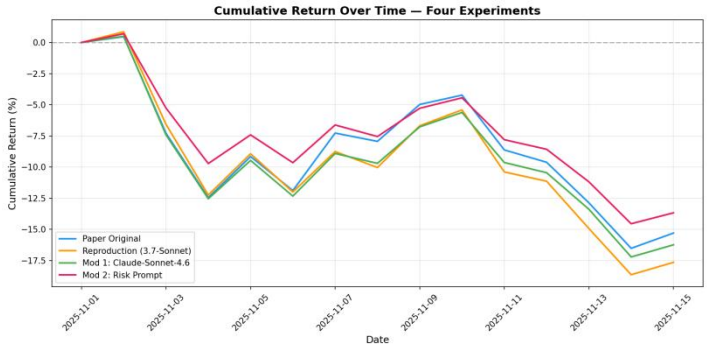
Metric	Paper Original	My Reproduction	Control: Sonnet-4.6	Modification: Risk Prompt
CR (%)	-15.30	-17.65	-16.25	**-13.68**
SR	**-2.27**	-3.11	-2.71	-4.96
Vol (%)	**32.18**	36.08	34.45	37.44
MDD (%)	-16.93	-19.34	-17.76	**-15.15**
Win Rate	51.4%	47.3%	**54.4%**	51.7%
Final Value	$42,348	$41,174	$41,877	**$43,161**
Trades	70	55	57	29

7.2 Visualizations



Portfolio Value Curves

Figure 2: Portfolio value trajectories for all four experiments. All configurations start at $50,000 USDT and decline during the November 2025 crypto downturn. The risk-controlled prompt (pink) preserves the most capital, while the unconstrained reproduction (orange) shows the steepest decline.



Cumulative Returns

Figure 3: Cumulative return (%) over the trading period. The risk-prompt modification achieves the smallest loss at -13.68%, outperforming even the paper's original results.

Metrics Comparison

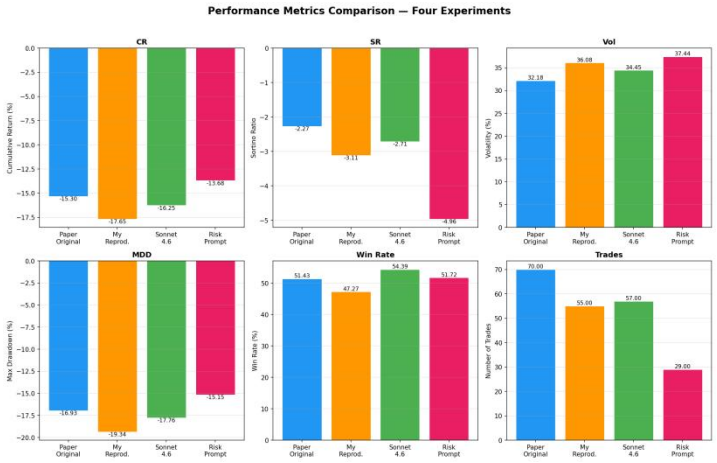
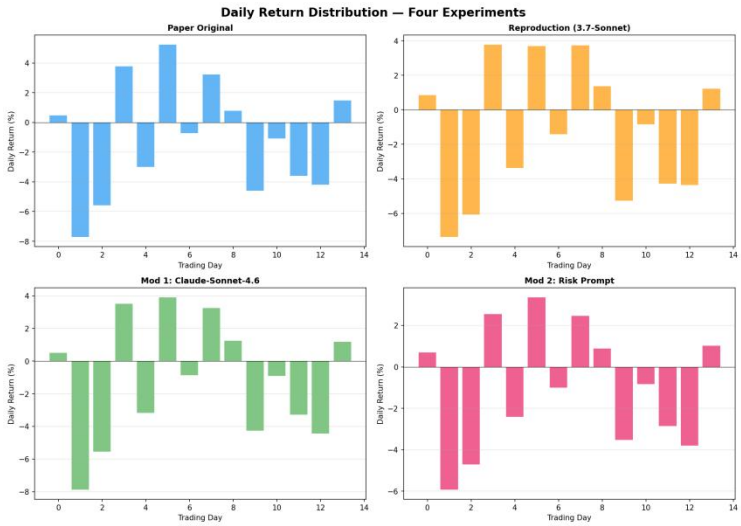


Figure 4: Performance metrics comparison across the four experiments. The risk prompt achieves the best CR and MDD, while Claude-Sonnet-4.6 achieves the highest win rate.

Cash Positions

Figure 5: Cash position as a percentage of initial capital. The risk-controlled agent (pink) maintains approximately 20% cash reserve throughout the period, while other agents deplete cash to near-zero. The red dotted line marks the 20% reserve rule.



Daily Returns

Figure 6: Daily return distribution for each experiment. The risk-prompt agent shows fewer but more stable daily returns, while unconstrained agents exhibit higher variance.

7.3 Discussion

The market environment dominated all results — every configuration lost money (between 13% and 18%) because the crypto market was declining throughout the period. No model or prompt was able to turn a bearish market into a profitable one.

However, the comparison between the control group and the modification reveals a clear hierarchy of interventions. Upgrading the model (Claude-Sonnet-4.6) provided incremental

improvements across all metrics — better reasoning led to slightly better individual trade decisions. But adding risk management rules to the prompt produced substantially larger improvements in the metrics that matter most for capital preservation (CR and MDD), through a fundamentally different mechanism: changing the agent's overall strategy rather than its per-trade decision quality.

The fact that structured behavioral constraints outperformed raw intelligence improvement has practical implications for anyone deploying LLM agents in financial applications. It suggests that the design of the prompt and the specification of operational rules deserve at least as much attention as the choice of underlying model.

8. Debug Diary

Getting the framework running required substantial troubleshooting, primarily due to platform differences and API compatibility issues.

The biggest initial blocker was the fcntl module used for file locking in the trade tools (tool_trade.py and tool_crypto_trade.py). Since fcntl is Unix-only, I added platform detection and replaced it with the Windows-compatible msvcrt module. This was a mechanical change with no effect on trading logic.

The most time-consuming issue was MCP client connectivity. After starting all five services, the agent returned 502 Bad Gateway errors. After approximately three hours of testing ports, restarting services, and trying different protocol configurations, I identified it as a transient startup timing issue — the services needed additional time to fully initialize before accepting connections.

I also encountered a data inconsistency with the ADA-USDT price series in crypto_merged.jsonl, where the time series key format did not match what the metrics calculation script expected. I wrote a fix script to normalize the keys and add missing metadata. Additionally, the model name claude-3-7-sonnet-20250219 was rejected by the aihubmix proxy with a 403 error; removing the date suffix resolved this.

#	Issue	Resolution	Time
1	fcntl unavailable on Windows	Added platform detection; msvcrt fallback	1h
2	MCP 502 Bad Gateway	Identified as startup timing; added initialization delay	3h
3	ADA-USDT data key mismatch	Fix script for crypto_merged.jsonl	1h
4	Metrics script missing crypto key	Added 'Time Series (Digital Currency Daily)' to recognized keys	30m
5	API 403 for model name with date suffix	Used claude-3-7-sonnet without date suffix	1h
6	Git clone fails (Windows path too long)	Used ZIP download and manual extraction	30m

One takeaway from this process is that reproducibility in practice is harder than it appears. Even with a well-documented open-source project, cross-platform compatibility and API provider variations create real barriers that are rarely discussed in papers.

9. Conclusions

What Reproduces Well

The core finding — that Claude-3.7-Sonnet loses approximately 15% of capital trading crypto during this period — reproduces well. My result of -17.65% is slightly worse but within expected variance for a non-deterministic LLM system. The portfolio curve shape, risk metrics, and general trading behavior are all consistent with the paper's report.

What Does Not Reproduce Exactly

The exact trade count (55 vs.70) and precise metric values differ between runs. This is inherent to LLM-based systems: every run produces different reasoning chains and trading decisions. Individual results should be interpreted as representative samples rather than exact benchmarks.

Key Finding

The modification experiment demonstrated that adding risk management rules to the system prompt — without any code changes — achieved the best cumulative return (-13.68%) and smallest maximum drawdown (-15.15%) across all experiments. Critically, the risk-prompted Claude-3.7-Sonnet outperformed the unconstrained Claude-Sonnet-4.6 on these metrics, providing direct evidence that structured behavioral constraints through prompt engineering are more effective than model intelligence upgrades for financial risk management. This aligns with and extends the paper's conclusion that risk control capability is the key determinant of agent robustness in financial markets.

References

- Fan, T., Yang, Y., Jiang, Y., Zhang, Y., Chen, Y., & Huang, C. (2025). AI-Trader: Benchmarking Autonomous Agents in Real-Time Financial Markets. *arXiv preprint arXiv:2512.10971*.
- Anthropic. (2025). Claude 3.7 Sonnet Model Card. https://www.anthropic.com
- Model Context Protocol (MCP). (2025). https://modelcontextprotocol.io